

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	S. Dharshini	Reg. No.:	192511260
Section / Batch:	2025-2nd year	Date:	20/07/2026

Code of Conduct

I S. Dharshini (192511260) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Dharshini

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

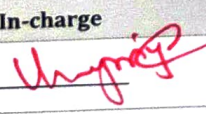
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

26	192524066	RAGUL M	1. Identify if the given expressions are balanced or not. (i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100]$, (ii) $((a*b) + [u/2] - d)$. 2. Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations. Apply stack ADT modifications. Analyze trade-off between time and space
27	192511260	S DHARSHINI	1. A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers. 2. Design a system that dynamically chooses between linear search and binary search based on: ➤ Data size ➤ Sortedness ➤ Frequency of queries Analyze decision criteria mathematically
28	192471038	SASI KUMAR MEGHA	1. Insert the following elements 100, 200, 300, 400, 500 into the linked stack and pop the elements 100 and 200 from the linked stack. Show the linked structure before and after linked stack operations. 2. An online food delivery service receives the following orders: O1, O2, O3, O4. While O2 is being prepared, an express order O5 arrives. <i>Questions:</i> ➤ Explain why a normal queue may not satisfy customer requirements. ➤ Recommend an appropriate queue implementation and justify your choice.
29	192424359	SUNDARESAN R U	1. Construct an efficient system using two stacks within a single array structure to maximize memory utilization. Compare fixed partitioning and dynamic allocation strategies, and analyze how each approach affects overflow conditions, space efficiency, and operational complexity under varying workloads. 2. A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning.
30	192524479	SHAHANA	1. Convert the following infix expression into postfix notation. Show the stack contents after processing each symbol. $((A+B)*C-(D-E)*(F+G))/H$ 2. For the expression. $((8+4)*(7-3)/(2+1))+(5*(6-2))$ Perform the following:

38	192524251	VAISHNAV M	1. Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. 2. A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. ➤ Analyze why this occurs. ➤ Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	1. Consider the singly linked list: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$. Delete the node containing 30. Explain how the links change after deletion. 2. Given two sorted linked lists: L1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ L2: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

C O 2 - ATI - ANALYTICAL PROBLEM

Name: S. Dhorevina

Reg. No.: 192511265

Course: C8A0305
Data Structures
Course code: C8A0305

1. CIRCULAR QUEUE

Definition :-

A circular queue is a linear data structure that follows the FIFO (First In First Out) principle. Unlike a linear queue, the last position is connected to the first position, allowing efficient utilization of memory by reusing empty spaces.

Operations Given:-

- Enqueue (10)
- Enqueue (20)
- Enqueue (30)
- Dequeue ()
- Enqueue (40)
- Enqueue (50)
- Enqueue (60)
- Enqueue (70)

Queue Size = 6

Initial state: [-, -, -, -, -, -]

Front = -1

Rear = -1

Step 1: Enqueue(10)

Since the queue is empty, both Front and Rear become

Index: 0 1 2 3 4 5

Queue: 10 - - - -

Front = 0

Rear = 0

Step 2: Enqueue(20)

Rear moves to the next position.

Index: 0 1 2 3 4 5

Queue: 10 20 - - -

Front = 0

Rear = 1

Step 3: Enqueue(30)

Index: 0 1 2 3 4 5

Queue: 10 20 30 - -

Front = 0

Rear = 2

Step 4: Dequeue()

The first element (10) is removed.

Index: 0 1 2 3 4 5

Queue: - 20 30 - -

Front = 1

Rear = 2

Step 5: Enqueue(40)

Index: 0 1 2 3 4 5

Queue: - 20 30 40 -

Front = 1

Rear = 3

Step 6: Enqueue(50)

Index: 0 1 2 3 4 5
Queue: - 20 30 40 50 -
Front = 1
Rear = 4

Step 7: Enqueue(60)

Index: 0 1 2 3 4 5
Queue: - 20 30 40 50 60

Front = 1

Rear = 5

Step 8: Enqueue(70)

Since the queue is circular, Rear wraps around from index 5 to index 0.

Index: 0 1 2 3 4 5
Queue: 70 20 30 40 50 60

Front = 1

Rear = 0

Final Queue

20 → 30 → 40 → 50 → 60 → 70

Final Pointer Values

Front = 1

Rear = 0

Circular Queue Algorithm

Enqueue

If $(Rear + 1) \% Size == Front$

Queue Overflow

ELSE IF Queue is Empty

Front = Rear = 0

ELSE

Rear = $(Rear + 1) \% Size$

Insert Element.

Dequeue

IF Queue is Empty

Queue Underflow

ELSE IF $\text{front} == \text{Rear}$

Delete Element

$\text{front} = \text{Rear} = -1$

ELSE

Delete Element

$\text{front} = (\text{front} + 1) \% \text{Size}$

Time Complexity

Operation	Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Peek	$O(1)$

Applications

- CPU Scheduling
- Printer Queue
- Network Packet Buffer
- Keyboard Buffer

2. Dynamic Selection Between Linear Search and Binary Search

Introduction:-

Searching is one of the most fundamental operations in Computer Science. Choosing the correct search algorithm significantly affects execution time. A dynamic system selects the most suitable algorithm depending on:-

- Data Size
- Sortedness
- Frequency of Queries

Linear Search

Linear Search checks each element one by one until the desired element is found.

Algorithm:-

```

for i = 0 to n-1
    if A[i] == key
        return Found
return Not Found
  
```

Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

Binary Search

Binary Search repeatedly divides the sorted array into 2 halves.

Algorithm

```

low = 0
high = n-1
while (low <= high)
{
    mid = (low + high) / 2
    if (key == A[mid])
        Found
    else if (key < A[mid])
        high = mid - 1
    else
        low = mid + 1
}
  
```


Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log_2 n)$
Worst	$O(\log_2 n)$

Decision Criteria

1. Data size

For very small datasets ($n < 50$):

Linear Search.

Reason:-

- Less overhead
- Faster in practice.

For large datasets: Binary Search

2. Sortedness.

If array is sorted: Binary Search

If array is unsorted: Linear search

3. Frequency of Queries

If only one search is required

Linear Search

If many searches are required

Sort once + Binary search.

Mathematical Analysis

Let

n = Number of Elements

Q = Number of searches.

Linear search

For one search $T = n$

For Q searches $T = Q \times n$.

Binary search

Sorting cost
 $O(n \log_2 n)$

Searching cost: $- O(\log_2 n)$

$$\text{Total} = n \log_2 n + A \log_2 n$$

Example

Suppose

$$n = 1000$$

$$A = 200$$

Linear search

$$\begin{aligned} T &= 200 \times 1000 \\ &= 200000 \text{ comparisons.} \end{aligned}$$

Binary search

$$\log_2 1000 = 10$$

Sorting

$$1000 \times 10 = 10000$$

searching

$$200 \times 10 = 2000$$

~~Total~~

~~12000 operations~~

Since

$$12000 \ll 200000$$

Binary search is much faster.

Urging